



DATABASE & SQL

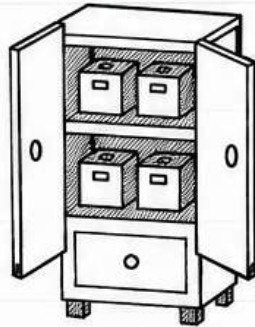
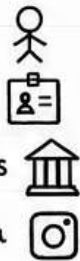


1. WHAT IS DATABASE?

A **Database** is an organized collection of data that can be easily stored, managed and accessed electronically.

Examples:

- Student records
- Employee details
- Banking transactions
- Instagram user data



Real-world example:

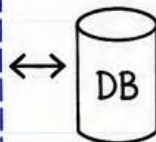
Think of a database like a digital cupboard where information is stored in a structured way. 😊

2. WHAT IS SQL?

SQL (**Structured Query Language**) is a language used to communicate with databases.

Using SQL, we can:

- ✓ Store data
- ✓ Retrieve data
- ✓ Update data
- ✓ Delete data
- ✓ Create tables
- ✓ Create databases



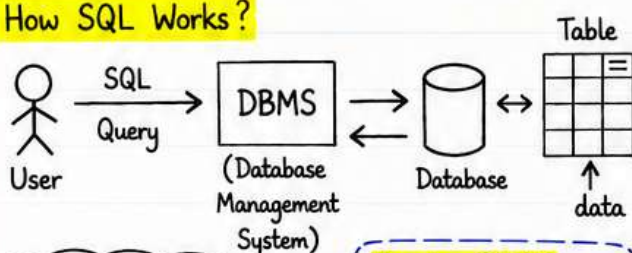
Example SQL Query:

```
SELECT * FROM students;
```

* means all columns

→ This query fetches all records from the 'students' table.

How SQL Works?



DBMS acts as an intermediary between user and database. ★

Popular DBMS:

- MySQL
- Oracle
- PostgreSQL
- Microsoft SQL Server

3. TYPES OF DATABASE

① Relational Database (RDBMS)

Data is stored in tables (rows & columns),

Examples: MySQL, Oracle, PostgreSQL, Microsoft SQL Server.

Features: Uses SQL, maintains relationships between tables, highly structured.

Students Table

ID	Name	Age
1	Riya	20
2	Amit	21

related
using ID

Courses Table

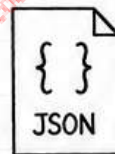
ID	Course
1	SQL
2	Python

② Non-Relational / NoSQL Database

Stores unstructured or semi-structured data.

Examples: MongoDB, Cassandra, Redis.

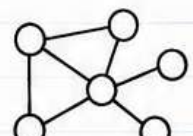
Features: Flexible schema, faster for large-scale applications, used in real-time apps.



Document

Key	Value
:	=

Key-Value

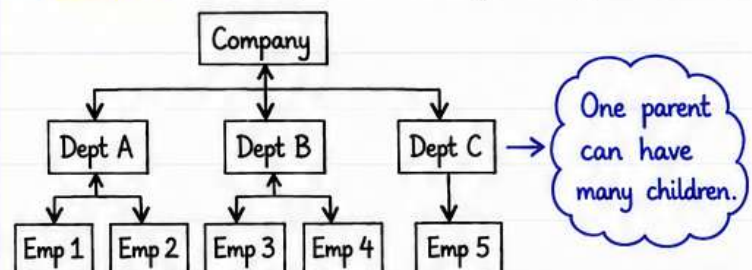


Graph

③ Hierarchical Database

Data is stored in a tree-like structure (Parent → Child).

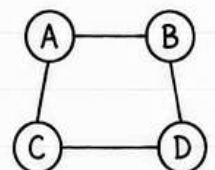
Example: IBM Information Management System (IMS)



④ Network Database

Data is connected like a network where multiple relationships exist.

Example: Integrated Data Store (IDS)



⑤ Object-Oriented Database

Stores data in the form of objects.

Example: ObjectDB



SQL SQL COMMANDS

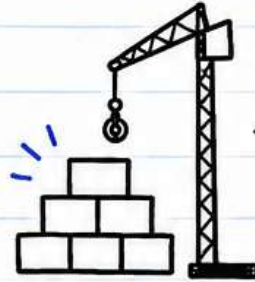
SQL (Structured Query Language) is used to communicate with databases. It is classified into **5** main types based on what they do. 😊

1 DDL - Data Definition Language

These commands define or modify the **structure** of the database (tables, schemas, etc.).

Examples:

- CREATE
- ALTER
- DROP
- TRUNCATE



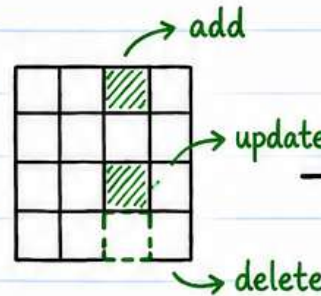
Think:
Structure
of the
database

2 DML - Data Manipulation Language

These commands deal with the **data** inside tables.

Examples:

- INSERT
- UPDATE
- DELETE



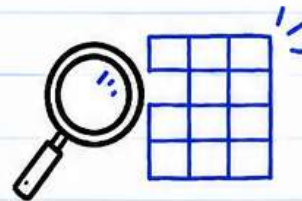
Think:
Changing
the data

3 DQL - Data Query Language

Used to **retrieve** data from the database.

Examples:

- SELECT



Think:
Getting
data

4 DCL - Data Control Language

These commands control **access** and **permissions**.

Examples:

- GRANT
- REVOKE



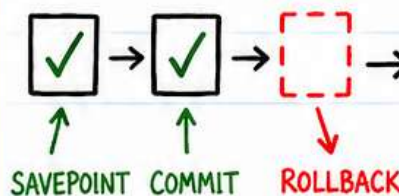
Think:
Who can
access
what

5 TCL - Transaction Control Language

Used to manage **transactions**. → (groups of SQL operations).

Examples:

- COMMIT
- ROLLBACK
- SAVEPOINT



Think:
Managing
changes
safely



Tip: DDL = Structure | DML = Data changes | DQL = Get data
DCL = Permissions | TCL = Transactions | 😊



DB

SQL QUERIES

SQL (Structured Query Language) → used to manage and manipulate data in a database.

1 CREATE → Create a New Table

```
CREATE TABLE student (  
  id INT,  
  name VARCHAR(50),  
  age INT  
);
```

student (table)

id	name	age

2 INSERT → Add Data into Table

```
INSERT INTO student (id, name, age)  
VALUES (1, 'Virat', 37);
```

Think:

Create → Insert
→ Select (Read)
→ Update → Delete

3 SELECT → Fetch Data from Table

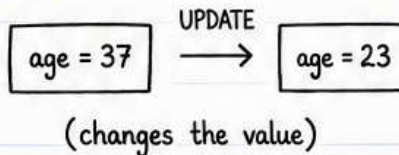
```
SELECT * FROM student; -- get all data  
SELECT * FROM student WHERE age > 20;
```

Data

id	name	age
1	Virat	37
...	...	...

4 UPDATE → Modify Existing Data

```
UPDATE student  
SET age = 23  
WHERE id = 1;
```



5 DELETE → Remove Data from Table

```
DELETE FROM student  
WHERE id = 1;
```

id	name	age
1	Virat	37
...	...	...

DELETE →

6 ALTER → Modify Table Structure

- Add Column : ALTER TABLE student
ADD email VARCHAR(100);
- Modify Column : ALTER TABLE student
MODIFY age INT;

Before			After Add Column	
id	name	age	id	name

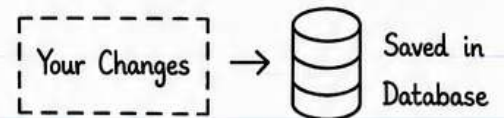
7 RENAME → Rename Table

```
RENAME TABLE student TO students;
```



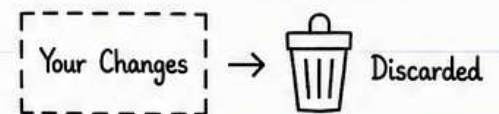
8 COMMIT → Save Changes Permanently

COMMIT;



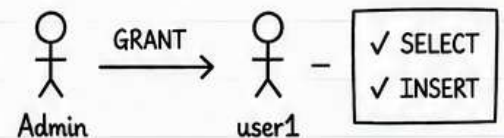
9 ROLLBACK → Undo Changes

ROLLBACK;



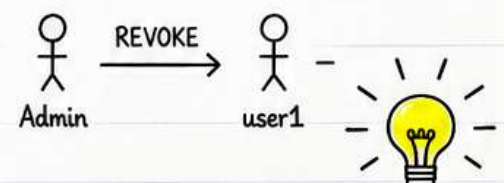
10 GRANT → Give Permissions

```
GRANT SELECT, INSERT  
ON student  
TO user1;
```



11 REVOKE → Remove Permissions

```
REVOKE INSERT  
ON student  
FROM user1;
```



PRIMARY KEY & FOREIGN KEY IN SQL

Keys in SQL are used to uniquely identify rows and link tables together.



1 PRIMARY KEY

A primary key is a column (or set of columns) that uniquely identifies each row in a table.

Key Features:

- Must contain unique values.
- Cannot contain NULL values.
- Only one primary key per table (but it can have multiple columns → **Composite Key**).

Example:

```
CREATE TABLE Students (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(50),  
  age INT  
);
```

student_id	name	age
1	Alice	20
2	Bob	21
3	Charlie	22

↑
Unique
(No Duplicates)

2 FOREIGN KEY

A foreign key is a column (or set of columns) in one table that refers to the primary key in another table.

Key Features:

- Can contain duplicate values.
- Can contain NULL values (unless restricted).
- Enforces referential integrity (ensures valid relationships).

Example:

```
CREATE TABLE Courses (  
  course_id INT PRIMARY KEY,  
  course_name VARCHAR(50)  
);
```

course_id	course_name
101	Math
102	Science
103	DBMS

```
CREATE TABLE Enrollments (  
  enrollment_id INT PRIMARY KEY,  
  student_id INT,  
  course_id INT,  
  FOREIGN KEY (student_id) REFERENCES Students(student_id),  
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

enrollment_id	student_id	course_id
1	1	101
2	2	102
3	1	103
4	3	101

↑
FK → Students
(student_id)

↑
FK → Courses
(course_id)

student_id is a
FOREIGN KEY
(refers to
Students.student_id)

course_id is a
FOREIGN KEY
(refers to
Courses.course_id)

PRIMARY KEY vs FOREIGN KEY

Feature 	Primary Key	Foreign Key
Purpose	Uniquely identify rows	Link tables together
Uniqueness	Must be unique	Can have duplicates
NULL Values	Not allowed	Allowed (usually)
Number per table	Only one	Multiple allowed



SQL CONSTRAINTS

Think:
Rules for
clean data!



→ Constraints are rules applied to **table columns** to ensure **data integrity** and **consistency**.

1 NOT NULL

Prevents **NULL** values in a column.

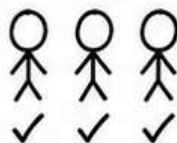
```
CREATE TABLE Students (  
  id INT,  
  name VARCHAR(100) NOT NULL.  
);
```



2 UNIQUE

Ensures all values in a column are **unique**.

```
CREATE TABLE Users (  
  email VARCHAR(100) UNIQUE  
);
```



It can be on multiple columns too!

```
CREATE TABLE Enrollments (  
  student_id INT,  
  course_id INT,  
  UNIQUE (student_id, course_id)  
);
```

3 PRIMARY KEY

Combines → **NOT NULL** + **UNIQUE**

```
CREATE TABLE Employees (  
  emp_id INT PRIMARY KEY,  
  name VARCHAR(100)  
);
```



Composite Primary Key:

```
CREATE TABLE Orders (  
  order_id INT,  
  product_id INT,  
  PRIMARY KEY (order_id, product_id)  
);
```

Primary Key
can never be
NULL or
Duplicate! ★

4 FOREIGN KEY

Creates a **relationship** between two tables.



```
CREATE TABLE Departments (  
  dept_id INT PRIMARY KEY,  
  dept_name VARCHAR(50)  
);
```

```
CREATE TABLE Employees (  
  emp_id INT PRIMARY KEY,  
  dept_id INT,  
  FOREIGN KEY (dept_id)  
  REFERENCES Departments(dept_id)  
);
```

Each employee
belongs to
one department.

5 CHECK

Restricts values based on a **condition**.

```
CREATE TABLE Products (  
  price DECIMAL(10,2),  
  CHECK (price > 0)  
);
```



Keeps bad
data out!

Multiple conditions:

```
CHECK (age >= 18 AND age <= 60)
```

6 DEFAULT

Sets a **default value** if none is provided.

```
CREATE TABLE Orders (  
  status VARCHAR(20)  
  DEFAULT 'Pending'  
);
```

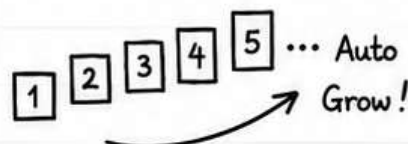


Default
gift! 😊

7 AUTO_INCREMENT / IDENTITY

Automatically generates sequential numeric values.

- MySQL : id INT **AUTO_INCREMENT**
- SQL Server : id INT **IDENTITY** (1,1)
- PostgreSQL : id **SERIAL**





SQL: DELETE vs TRUNCATE vs DROP

These are SQL commands used to remove data.

1 DELETE

- Used to remove **selected** rows.
- Can use **WHERE** condition.
- Table **structure** remains.

Syntax:

```
DELETE FROM students  
WHERE id = 1;
```

Example:

Before:

id	name
1	Ram
2	Shyam



```
DELETE FROM students  
WHERE id = 1;
```

After:

id	name
2	Shyam

Deletes only matching rows.



2 TRUNCATE

- Removes **all rows** from the table.
- Cannot use **WHERE**.
- Faster** than DELETE.
- Table **structure** remains.

Syntax:

```
TRUNCATE TABLE students;
```

Example:

Before:

id	name
1	Ram
2	Shyam



```
TRUNCATE TABLE students;
```

After:

id	name



All data deleted, Table still exists.

3 DROP

- Completely** removes the table from database.
- Data + Structure **both** deleted.

Syntax:

```
DROP TABLE students;
```

Example:

Before:

students			



```
DROP TABLE students;
```

After:

Table does not exist!



* Some databases may support recovery features for **TRUNCATE**, but generally it cannot be rolled back.



DROP is the fastest!



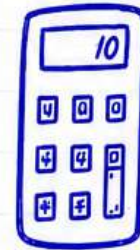
SQL OPERATORS

LEARN,
PRACTICE,
MASTER 😊

Operators are **special symbols/keywords** used to perform **operations on data** in a database.

① ARITHMETIC OPERATORS - used to perform mathematical calculations.

Operator	Description	Example
+	Addition	salary + 1000
-	Subtraction	salary - 500
*	Multiplication	salary * 2
/	Division	salary / 2
%	Modulus (remainder)	salary % 2



Do the
math in
SQL!

② COMPARISON (RELATIONAL) OPERATORS - used to compare two values.

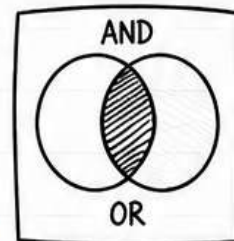
Operator	Description	Example
=	Equal to	age = 25
!= or <>	Not equal to	age != 30 (or age <> 30)
>	Greater than	salary > 50000
<	Less than	salary < 30000
>=	Greater than or equal	age >= 18
<=	Less than or equal	age <= 60



Compare &
filter data
easily!

③ LOGICAL OPERATORS - used to combine multiple conditions.

Operator	Description	Example
AND	All conditions true	age > 18 AND city = 'NY'
OR	Any condition true	age < 18 OR age > 60
NOT	Negates condition	NOT (city = 'London')



AND =
both
OR =
either
NOT =
opposite

④ SPECIAL OPERATORS - used for specific conditions.

BETWEEN	→ Checks if a value is within a range	10 X 20
IN	→ Matches any value in a list	('HR', 'IT', 'Finance')
LIKE	→ Searches for a pattern	'A%'
IS NULL	→ Checks for NULL values	NULL
EXISTS	→ Checks if a subquery returns results	🔍
ANY / ALL	→ Compare a value with a subquery	> ALL (...)

Powerful
operators for
advanced
filtering!



SQL CLAUSES

Building blocks of an SQL query!

1

SELECT

Specifies which columns you want to retrieve.

```
SELECT name, age FROM students;
```

students table

id	name	age
1	Anu	20
2	Ravi	21
3	Isha	19

→ We get only these columns

name	age
Anu	20
Ravi	21
Isha	19

2

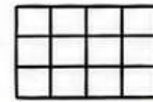
FROM

Indicates the table you're querying.

```
SELECT name, age FROM students;
```



Database



students (table)

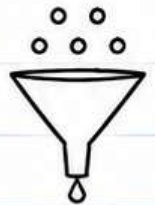
3

WHERE

Filters rows based on a condition.

```
SELECT name, age FROM students WHERE age > 18;
```

Only rows where age is greater than 18



4

GROUP BY

Groups rows that have the same values.
(Used with aggregate functions like **COUNT**, **SUM**)

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department;
```

Group same departments together



5

HAVING

Filters groups (used after **GROUP BY**).

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department  
HAVING COUNT(*) > 5;
```

Only groups with more than 5 employees

Result

IT	12
HR	8
Sales	6

→ 5 (threshold)

6

ORDER BY

Sorts the result set.

```
SELECT name, age FROM students ORDER BY age DESC;
```

ASC (small → big)

1, 2, 3, 4, ...

DESC (big → small)

4, 3, 2, 1, ...

21
20
19
18

7

JOIN

Combines rows from multiple tables.

```
SELECT students.name, courses.course_name  
FROM students  
JOIN courses ON students.course_id = courses.id;
```

students

id	name	course_id
1	Anu	101
2	Ravi	102

courses

id	course_name
101	SQL
102	Python

JOIN on course_id = id

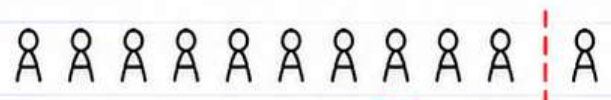
8

LIMIT

(or TOP)

Restricts the number of rows returned.

```
SELECT name, age FROM students LIMIT 10;
```



Return only first 10 rows

SQL AGGREGATE FUNCTIONS

- ★ Aggregate functions perform calculations on multiple rows of data and return a single value. They are commonly used with the GROUP BY clause.

1) COMMON SQL AGGREGATE FUNCTIONS

Function	Description	Example
COUNT()	Counts rows	COUNT(*)
SUM()	Adds values	SUM(salary)
AVG()	Calculates average	AVG(marks)
MIN()	Finds smallest value	MIN(price)
MAX()	Finds largest value	MAX(age)

These functions ignore NULL values (except COUNT(*))



2) SYNTAX

```
SELECT AGGREGATE_FUNCTION (column_name)
FROM table_name ;
```



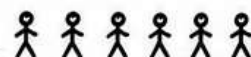
Always remember:
Aggregate function returns a **SINGLE VALUE** !

3) EXAMPLES

- i) COUNT() → Counts the number of employees.

```
SELECT COUNT(*) AS total_employees
FROM employees ;
```

→ returns 12



Counting rows

- ii) SUM() → Calculates total salary.

```
SELECT SUM(salary) AS total_salary
FROM employees ;
```

→ returns 850000



Adding up!

- iii) AVG() → Finds average salary.

```
SELECT AVG(salary) AS average_salary
FROM employees ;
```

→ returns 70833.33



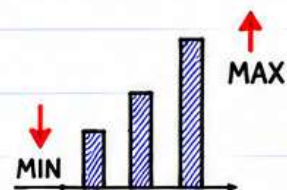
Average balance

- iv) MIN() and MAX() → Find lowest and highest salary.

```
SELECT MIN(salary) AS minimum_salary,
       MAX(salary) AS maximum_salary
FROM employees ;
```

→ min 30000

→ max 120000



4) USING AGGREGATE FUNCTIONS WITH GROUP BY

Suppose you want the average salary department-wise:

```
SELECT department,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY department ;
```

Group

by dept.

Avg

55000

75000

5) USING HAVING WITH AGGREGATES

HAVING filters grouped data (after aggregation).

Example: departments with average salary greater than 50000.

```
SELECT department,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000 ;
```



FILTER

SQL JOINS

JOINS are used to combine rows from two or more tables based on a related column.

TABLES WE WILL USE:

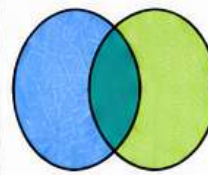
Customers

	customer_id	name
①	1	Alice
②	2	Bob
③	3	Charlie

Orders

	order_id	customer_id	product
	101	1	Laptop
	102	1	Mouse
	103	2	Keyboard

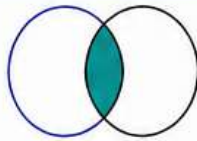
LEGEND (HOW TO READ DIAGRAMS)



- Left table (Customers)
- Right table (Orders)
- Both tables (match)
- No match (NULL)

1 INNER JOIN

Returns only matching rows from both tables.



Result →

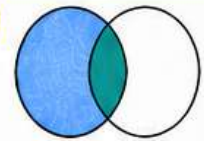
name	product
Alice	Laptop
Alice	Mouse
Bob	Keyboard

```
SELECT c.name, o.product
FROM Customers c
INNER JOIN Orders o
ON c.customer_id =
o.customer_id;
```

★ Only customers with orders appear.

2 LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table and matching rows from the right table.



Result →

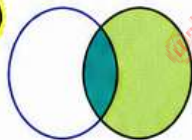
name	product
Alice	Laptop
Alice	Mouse
Bob	Keyboard
Charlie	NULL

```
SELECT c.name, o.product
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id =
o.customer_id;
```

★ All customers appear, even without orders.

3 RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table and matching rows from the left table.



Result →

name	product
Alice	Laptop
Alice	Mouse
Bob	Keyboard

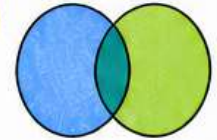
```
SELECT c.name, o.product
FROM Customers c
RIGHT JOIN Orders o
ON c.customer_id =
o.customer_id;
```

★ If an order had no matching customer, customer fields would be NULL.



4 FULL OUTER JOIN (FULL JOIN)

Returns all rows when there is a match in either table.



Result →

name	product
Alice	Laptop
Alice	Mouse
Bob	Keyboard
Charlie	NULL

```
SELECT c.name, o.product
FROM Customers c
FULL OUTER JOIN Orders o
ON c.customer_id =
o.customer_id;
```

★ Includes matching rows

★ Includes unmatched customers and unmatched orders



5 CROSS JOIN

Returns the Cartesian product (all combinations) of both tables.



Result → 3 x 3 = 9 rows

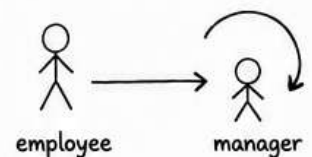
name	product
Alice	Laptop
Alice	Mouse
Alice	Keyboard
Bob	Laptop
Bob	Mouse
Bob	Keyboard
Charlie	Laptop
Charlie	Mouse
Charlie	Keyboard

```
SELECT c.name, o.product
FROM Customers c
CROSS JOIN Orders o;
```

★ If there are ③ customers and ③ orders, result will have ⑨ rows.

6 SELF JOIN

A table joins with itself.
Example: employees and managers.



Result →

Employee	Manager
John	Mary
Mike	John
Sara	John
Tom	Mary

```
SELECT A.name AS Employee,
B.name AS Manager
FROM Employees A
JOIN Employees B
ON A.manager_id = B.employee_id;
```

★ Matches rows within the same table.





TOP 5 SQL INTERVIEW QUERIES



Goal: Crack SQL rounds by mastering these must-know queries!

Sample Table: **Employee**

id	name	email	salary	dept
1	Alice	alice@x.com	80000	IT
2	Bob	bob@x.com	60000	HR
3	Carol	carol@x.com	90000	IT
4	Dan	dan@x.com	60000	HR
5	Eve	eve@x.com	75000	IT
...	...	...	...	...

We will use this table for all examples.

KEY REMINDERS



- SQL keywords are NOT case-sensitive.
- Semicolon (;) ends the query.
- Test on your DB (MySQL / PostgreSQL / SQL Server)

1 FIND 2ND HIGHEST SALARY

Method 1 (Subquery)

```
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (
    SELECT MAX(salary)
    FROM Employee
);
```

Salary
90000
80000
75000
60000
60000

← 2nd highest

Method 2 (Using DISTINCT + LIMIT)

```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Returns the 2nd highest distinct salary.

2 FIND NTH HIGHEST SALARY

Method 1 (MySQL - LIMIT)

```
-- For N = 3 (3rd highest)
SELECT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 2; -- N-1
```

General:
LIMIT 1
OFFSET (N-1)

Method 2 (Using DENSE_RANK())

```
SELECT salary FROM (
    SELECT salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS rn
    FROM Employee
) e
WHERE rn = (N);
```

Rank helps handle ties!



3 FIND DUPLICATE RECORDS

★ Based on a column (email)

```
SELECT email, COUNT(*)
FROM Employee
GROUP BY email
HAVING COUNT(*) > 1;
```

Let's find duplicates!



★ Duplicate complete records

```
SELECT name, salary, dept, COUNT(*)
FROM Employee
GROUP BY name, salary, dept
HAVING COUNT(*) > 1;
```

4 DELETE DUPLICATE RECORDS

Method 1 (Self Join) - keep smallest id

```
DELETE e1
FROM Employee e1
JOIN Employee e2
ON e1.email = e2.email
AND e1.id > e2.id;
```

Keeps the row with smallest id and deletes others.

Method 2 (Using ROW_NUMBER())

```
DELETE FROM Employee
WHERE id IN (
    SELECT id
    FROM (
        SELECT id,
        ROW_NUMBER() OVER
        (PARTITION BY email ORDER BY id) AS rn
    FROM Employee
    ) t
    WHERE rn > 1);
```



Remove duplicates

5 GET EVEN & ODD ROWS

Using MOD() (based on id)

Even Rows

```
SELECT * FROM Employee
WHERE MOD(id, 2) = 0;
```

Odd Rows

```
SELECT * FROM Employee
WHERE MOD(id, 2) <> 0;
```

Using ROW_NUMBER() (based on order)

Even Rows

```
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (ORDER BY id) AS rn
    FROM Employee
) t
WHERE rn % 2 = 0;
```

Odd Rows

```
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (ORDER BY id) AS rn
    FROM Employee
) t
WHERE rn % 2 = 1;
```

Use when id is not sequential or has gaps.





WHAT IS NORMALISATION?



It is the process of organizing data in a database to **reduce redundancy** and improve **data integrity** by dividing large tables into smaller related tables.

WHY NORMALISE?

- ★ Reduce duplicate data
- ★ Improve consistency
- ★ Easier updates & maintenance
- ★ Better data integrity

Good DB Design =
Less Redundancy
+ More Integrity

EXAMPLE - UNNORMALISED TABLE (BAD)

StudentID	StudentName	Course	Instructor
1	John	SQL	Smith
1	John	Python	David
2	Alice	SQL	Smith

PROBLEMS:

- ① Student names repeat
- ② Instructor names repeat
- ③ Updating instructor info requires multiple changes

Too much duplication!



NORMAL FORMS

① 1NF (First Normal Form)

- Each column contains **atomic** (single) values
- No repeating groups

✗ NOT IN 1NF

Student	Courses
John	SQL, Python

✓ IN 1NF

Student	Course
John	SQL
John	Python

Atomic = indivisible value



③ 3NF (Third Normal Form)

- Must be in 2NF
- No **transitive dependency**.

EXAMPLE TABLE (IN 2NF)

EmpID	EmpName	DeptID	DeptName
1	Rahul	10	HR
2	Priya	20	IT

DeptName depends on DeptID, not directly on EmpID

SOLUTION → Split into 2 tables

EMPLOYEES

EmpID	EmpName	DeptID
1	Rahul	10
2	Priya	20

DEPARTMENTS

DeptID	DeptName
10	HR
20	IT

EmpID → DeptID → DeptName
(Transitive Dependency)

② 2NF (Second Normal Form)

- Must be in 1NF
- No **partial dependency**
- Non-key attributes must depend on the **whole** primary key

StudentName depends only on StudentID

EXAMPLE TABLE (IN 1NF)

StudentID	CourseID	StudentName	CourseName
1	101	John	SQL
1	102	John	Python

CourseName depends only on CourseID

SOLUTION → Split into 3 tables

STUDENTS

StudentID	StudentName
1	John
2	Alice

ENROLLMENT

StudentID	CourseID
1	101
1	102
2	101

COURSES

CourseID	CourseName
101	SQL
102	Python

④ BCNF (Boyce-Codd Normal Form)

- A stricter version of 3NF
- Every **determinant** must be a **candidate key**.

NOTE:

Used in advanced database design to handle complex relationships.



EXAMPLE SQL STRUCTURE (AFTER NORMALISATION)

```
CREATE TABLE Students (
  StudentID INT PRIMARY KEY,
  StudentName VARCHAR(100)
);
```

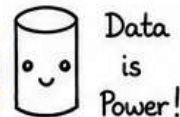
```
CREATE TABLE Courses (
  CourseID INT PRIMARY KEY,
  CourseName VARCHAR(100)
);
```

```
CREATE TABLE Enrollment (
  StudentID INT,
  CourseID INT,
  PRIMARY KEY (StudentID, CourseID),
  FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
  FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
```

Well structured DB = Easy to maintain!



DBMS: VIEW, TRANSACTION & ACID PROPERTIES

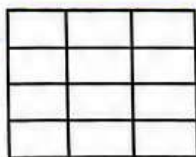


1. VIEW in DBMS

→ A **VIEW** is a virtual table created from one or more tables using a SQL query.

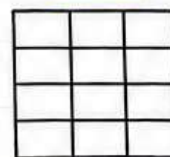
It does **NOT** store data itself; it displays data stored in underlying tables.

Base Table



SQL QUERY

VIEW



No Data Stored
in View

Why
use
VIEW?

- ① Improves **security** (restrict access to specific columns/rows)
- ② **Simplifies** complex queries
- ③ Provides **data independence**

SYNTAX:

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

EXAMPLE:

```
CREATE VIEW emp_view AS
SELECT emp_id, emp_name, salary
FROM employee
WHERE department = 'IT';
```

2. TRANSACTION in DBMS

→ A **TRANSACTION** is a sequence of one or more SQL operations executed as a single unit of work.

Example: Money Transfer



Either both updates happen (Commit)
or none happen (Rollback) ★

After Transfer → A = ₹4000, B = ₹4000

SQL EXAMPLE:

```
BEGIN; ①
UPDATE account ②
SET balance = balance - 1000
WHERE acc_no = 101

UPDATE account ③
SET balance = balance + 1000
WHERE acc_no = 102

COMMIT; ④
```

TRANSACTION COMMANDS

- ① **BEGIN / START TRANSACTION** - Starts a transaction.
- ② **COMMIT** - Saves changes permanently.
- ③ **ROLLBACK** - Undoes changes.
- ④ **SAVEPOINT** - Creates a checkpoint within a transaction.

3. ACID PROPERTIES

→ **ACID** properties ensure reliable transaction processing. ★

A	ATOMICITY	A transaction is treated as a single unit. Either all operations succeed or <u>none</u> do.	Step1 → Step2 → Step3	All ✓ or None X
C	CONSISTENCY	The database remains in a <u>valid</u> state before and after the transaction. Rules and constraints are preserved.	→	Valid State Always!
I	ISOLATION	Multiple transactions can execute concurrently without affecting each other. Prevents data inconsistency.	T1 T2	No Interference
D	DURABILITY	Once a transaction is committed, changes are <u>permanently</u> stored. Data remains safe even after crash.	→ COMMIT →	Permanent & Safe! 😊